Architecture and Implementation of a High-Resolution Bit-Perfect Audio Engine for Android: Analysis of the Samsung Galaxy S24+ Platform and Dedicated Digital Audio Player Systems

Executive Summary and Technical Feasibility

Extracting deterministic, bit-perfect, ultra-low-jitter digital audio from an Android platform requires navigating an operating system architecture primarily optimized for multi-source concurrency rather than audiophile transparency. Android aggregates disparate, asynchronous audio sources—including notification pings, navigation alerts, voice telephony, and multimedia playback—through a centralized mixing pipeline. The core of this pipeline, AudioFlinger, enforces uniform sampling rates and formats, applies platform-level dynamic range compression, and routes streams through hardware-specific post-processing layers. On commercial flagship smartphones such as the Samsung Galaxy S24+ (Exynos 2400 platform, model SM-S926B) running One UI, these framework constraints are further compounded by proprietary audio processing modules, including SoundAlive, Adapt Sound, and Dolby Atmos. These layers inject non-linear equalization, stereo expansion, and volume-dependent dynamic filtering into the digital signal path.

Standard Android audio APIs, such as AudioTrack, OpenSL ES, and shared-mode AAudio, route digital audio through the platform mixer, which resamples streams to a fixed hardware rate (typically 48 kHz). While Android 14 (API level 34) introduced the AudioManager.setPreferredMixerAttributes() interface alongside AudioMixerAttributes.MIXER_BEHAVIOR_BIT_PERFECT to permit framework-level mixer bypass, practical deployment across commercial devices remains uneven. The underlying vendor Hardware Abstraction Layer (HAL) implementations frequently fail to handle arbitrary non-standard clock rates (such as 88.2 kHz, 176.4 kHz, or 352.8 kHz), trigger audio daemon crashes, or drop connections when handling rapid format transitions.

To achieve deterministic, bit-for-bit high-resolution playback on the Galaxy S24+ across the full spectrum of Pulse Code Modulation (PCM) rates (44.1 kHz to 768 kHz) and Direct Stream Digital (DSD) formats, an application must use a dedicated, user-space USB Audio Class (UAC2) driver. By requesting direct USB device access via the Android USB Host API (android.hardware.usb.UsbManager) and obtaining the underlying file descriptor through UsbDeviceConnection.getFileDescriptor(), the application can transfer control to an embedded C++ engine. This engine issues asynchronous isochronous transfers directly through the Linux USB filesystem (usbfs), bypassing AudioFlinger, the Android Audio HAL, and all Samsung One UI post-processing effects.

Wireless transmission across Bluetooth Advanced Audio Distribution Profile (A2DP) connections cannot achieve genuine bit-perfect fidelity on this hardware. Standard codecs, such as SBC, AAC, aptX, and LDAC, employ lossy psychoacoustic compression to fit within Bluetooth radio

bandwidth constraints. Furthermore, the Exynos 2400 chipset does not support Qualcomm Snapdragon Sound or aptX Lossless transmission. Consequently, an advanced Android audio architecture must separate wired and wireless pipelines: a bit-perfect, user-space driver pipeline for external USB-C DACs, and an optimized, dithered 24-bit floating-point processing engine for Bluetooth transmission.

Android Core Audio Architecture and Signal Processing Topologies

The Android audio subsystem operates as a multi-tier hierarchy spanning user-space application runtimes, native framework libraries, system servers, hardware abstraction layers, and Linux kernel drivers. Understanding the points at which digital audio data is modified requires tracing the signal pipeline from container parsing to hardware conversion.

```
+-------------------------------------------------------------------------
-------------------------------------------------+
|                                    DETAILED SYSTEM PIPELINE
COMPARISON MATRIX                                          |
+--------------------------+-----------------------+-----------------
------+----------------------+-----------------------+
| Signal Pathway           | Resampling Point      | Mixing Point
| DSP & Effects Point | HAL Interface         |
+--------------------------+-----------------------+-----------------
------+----------------------+-----------------------+
| A. Internal Speaker      | AudioFlinger SRC      | AudioFlinger
| SoundAlive / Dolby  | AIDL / HIDL Vendor    |
|                          | (e.g. 44.1 -> 48 kHz) | AudioMixer
| Speaker Protection  | Audio HAL             |
|                          |                       |
|                     |                       |
| B. Analog Jack / Dongle  | AudioFlinger SRC      | AudioFlinger
| Adapt Sound /       | AIDL / HIDL Vendor    |
|                          | (locks to 48 kHz)     | AudioMixer
| System Equalizer    | Audio HAL             |
|                          |                       |
|                     |                       |
| C. Standard USB DAC      | AudioFlinger SRC      | AudioFlinger
| Dolby Atmos         | Android USB Audio HAL |
|                          | (fixed HAL rate)      | AudioMixer
| (if enabled)        | (snd-usb-audio)       |
|                          |                       |
|                     |                       |
| D. Bluetooth A2DP        | AudioFlinger SRC      | AudioFlinger
| System Equalizer    | BluetoothAudioHAL /   |
|                          | (codec-dependent rate)| AudioMixer
| (pre-encoding)      | Codec Shared Library  |
|                          |                       |
|                     |                       |
```

```
| E. Custom User-Space USB | None                    | None
| None                  | None                    |
|    Audio Driver          | (Bit-Perfect)          | (Direct
Isochronous)    | (Bit-Perfect)       | (Direct usbfs ioctl)   |
+--------------------------+-------------------------+-----------------
------+----------------------+-----------------------+
```

Container Parsing, Decoding, and PCM Buffering

Audio playback begins at the storage boundary. Files formatted in lossless containers (such as FLAC, ALAC, WAV, AIFF, or DSF/DFF) reside in internal Universal Flash Storage (UFS) or accessible external storage. The application layer parses the container headers, extracts metadata, identifies frame boundaries, and delivers raw bitstreams to software decoders. The decoding engine—typically implemented in native C/C++ via libraries like libFLAC, FFmpeg (libavcodec), or libwavpack—converts the bitstream into uncompressed PCM samples. In standard implementations, these decoded samples are structured as interleaved signed integers (16-bit or 24-bit) or IEEE 754 single-precision 32-bit floats. Any buffer underrun or metadata parsing error at this boundary propagates downstream as timing jitter or audible dropped frames.

Framework Audio Interfaces: AudioTrack, AAudio, and OpenSL ES

Applications interact with the operating system's audio infrastructure using three primary interfaces:
The Java and Kotlin AudioTrack API represents the foundational audio interface in Android. It communicates across process boundaries via Binder Inter-Process Communication (IPC) to transfer audio buffers to the native AudioTrack client within audioserver using shared memory (ashmem) circular queues.
OpenSL ES (Open Sound Library for Embedded Systems) is a legacy native C interface that acts as an abstraction wrapper around the native AudioTrack client. Modern Android runtimes map OpenSL ES calls directly to the same internal queues used by Java AudioTrack, offering no latency or routing advantages.
AAudio, introduced in Android 8.0 (API level 26), is an audio interface written in C designed for high-performance applications. It operates in two sharing modes:

- *Shared Mode* (AAUDIO_SHARING_MODE_SHARED): Routes audio frames through AudioFlinger, subjecting the stream to the standard platform mixer.
- *Exclusive Mode* (AAUDIO_SHARING_MODE_EXCLUSIVE): Uses memory-mapped I/O (tryMMap = true) via AAudioServiceStreamMMAP to communicate directly with hardware-supported Audio HAL ring buffers, bypassing AudioFlinger.

However, MMAP exclusive mode requires explicit driver and HAL support from the hardware vendor. While smartphone vendors frequently implement MMAP for onboard speakers and microphones to reduce latency for system interactions, they rarely implement it for external USB endpoints, causing requests to fall back to shared mixing mode.

AudioFlinger, Resampling Algorithms, and Mixing Dynamics

AudioFlinger is the core audio service running within the audioserver daemon. It manages

multiple thread topologies inherited from PlaybackThread, including MixerThread, DirectOutputThread, and OffloadThread. When multiple applications play audio concurrently, MixerThread uses the AudioMixer class to combine incoming streams into a single output buffer. If an incoming audio stream does not match the configured hardware sample rate of the target output—which is almost universally fixed at 48 kHz on standard mobile configurations—AudioMixer invokes its internal Sample Rate Converter (SRC). The platform employs a multi-tiered resampling implementation: a fast linear interpolator, a cubic interpolator, and a band-limited polyphase sinc resampler adapted from the open-source Speex DSP engine. Even when using polyphase sinc interpolation, non-integer sample rate conversion (such as converting 44.1 kHz Red Book CD audio to 48 kHz) introduces mathematical interpolation artifacts. The conversion causes passband ripple, high-frequency phase alteration, and attenuation near the Nyquist cutoff frequency (f_s / 2).
Furthermore, AudioFlinger promotes all incoming integer formats to 32-bit floating-point for internal gain adjustment and mixing, and then quantizes the output back to integer format without applying Triangular Probability Density Function (TPDF) dither, which can introduce low-level quantization distortion.

AudioPolicyService and the Hardware Abstraction Layer

The AudioPolicyService evaluates active application audio attributes, checks output profiles defined in the system's /vendor/etc/audio_policy_configuration.xml file, and configures hardware routes. When an output path is selected, AudioPolicyService directs AudioFlinger to instantiate an output stream via the Audio HAL.
Modern Android platforms implement the Audio HAL using AIDL (Android Interface Definition Language) or HIDL (Hardware Interface Definition Language) HAL interfaces, which decouple vendor driver modules from the core framework. The vendor HAL communicates directly with Linux ALSA (Advanced Linux Sound Architecture) kernel devices via /dev/snd/pcmC*D*p control and streaming nodes.

Detailed Signal Pathways

- *Pathway A (Internal Speakers)*: A lossless FLAC file is decoded to 44.1 kHz, 16-bit integer PCM and passed to AudioTrack. AudioFlinger's MixerThread resamples the stream to 48 kHz using polyphase sinc interpolation and converts it to a 32-bit float. Samsung SoundAlive and Dolby Atmos insert speaker-tuning equalization, dynamic range compression (DRC), and stereo widening. The audio HAL quantizes the stream and passes it via ALSA to the Exynos PMIC/audio codec. Integrated smart amplifiers (e.g., Cirrus Logic CS35L series) apply thermal and excursion protection DSP before the internal speakers convert the signal to analog sound.
- *Pathway B (Analog Headphone Adapters)*: Decoded PCM data is passed through AudioTrack into AudioFlinger, where it is resampled to 48 kHz and processed through system EQ and Samsung Adapt Sound. The HAL routes the mixed stream to a DAC. If using a passive USB Type-C adapter relying on USB Audio Accessory Mode, the phone's internal codec handles conversion; if using an active adapter with an integrated DAC, the digital stream is converted in the dongle's integrated chip.
- *Pathway C (Standard Android USB Route)*: High-resolution 96 kHz, 24-bit PCM is written to AudioTrack. Because standard Android audio policy configurations define a single, fixed sample rate for the USB Audio HAL—typically 48 kHz—AudioFlinger resamples the

96 kHz stream down to 48 kHz. Active system audio effects remain inserted in the pipeline. The stream is then transferred via the kernel snd-usb-audio driver across USB Isochronous OUT endpoints.

- *Pathway D (Bluetooth A2DP)*: Decoded audio is passed to AudioFlinger, which resamples it to match the Bluetooth stack's negotiated rate (e.g., 44.1 kHz, 48 kHz, or 96 kHz) and applies system EQ curves. The stream passes through the BluetoothAudio[span_3](start_span)[span_3](end_span)HAL to the Bluetooth stack daemon (Fluoride/Gabeldorsche), which feeds an active codec encoder, such as LDAC or AAC. The encoded bitstream is framed into A2DP transport packets, transmitted across the 2.4 GHz radio link via HCI UART, and decoded by the receiving headphones' onboard DAC.
- *Pathway E (Custom User-Space USB Driver)*: High-resolution 192 kHz, 24-bit PCM or native DSD256 bitstreams are decoded inside the application's native C++ engine. The audio completely bypasses AudioTrack, AAudio, AudioFlinger, and the system Audio HAL. The application claims the USB interface using a Linux usbfs file descriptor and submits asynchronous isochronous USB Request Blocks (URBs) directly to the kernel USB controller. The hardware DAC receives the original digital samples, automatically matches its internal clock to 192 kHz, and converts the stream to analog without system interference.

Samsung Galaxy S24+ (Exynos 2400) Platform and Audio Subsystem Assessment

The international variant of the Samsung Galaxy S24+ (SM-S926B) is built on the Samsung Exynos 2400 system-on-chip (S5E9945), fabricated on a 4nm (4LPP+) process. The platform integrates a deca-core CPU subsystem, an AMD RDNA3-based Xclipse 940 GPU, and an integrated audio Digital Signal Processor dedicated to low-power wake-word detection, conversational speech processing, and system acoustic management. Teardowns and silicon-level analyses confirm that internal speaker amplification is driven by smart power amplifiers, such as the Cirrus Logic CS35L series, which feature hardware protection limiters and current-sense feedback.

```
+-------------------------------------------------------------------------
-------------------------------------------------+
|                                  SAMSUNG GALAXY S24+ (EXYNOS 2400)
CAPABILITY MATRIX                                  |
+--------------------------+----------------------+------------------
----------+--------------------+-------------------+
| Audio Capability         | Platform Support     | Source of
Evidence          | Implementation     | Verification      |
|                          |                      |
| Limitations        | Method            |
+--------------------------+----------------------+------------------
----------+--------------------+-------------------+
| Bit-Perfect USB via      | Supported            | Verified via
third-party    | Requires custom    | Hardware USB DAC  |
| Direct User-Space Driver |                      | apps (UAPP,
Neutron)        | user-space driver  | display; Digital  |
```

		and usbfs ioctl architecture	implementation	loopback null test
Native Android 14+	Highly Unreliable /	Issue trackers, Oboe test	HAL configuration	dumpsys media.
AudioMixerAttributes	Limited Support	reports on OEM builds	failures; crashes	audio_flinger;
Bit-Perfect Mode		[span_116](start_span)[span_116](end_span)[span_119](start_span)[span_119](end_span)	on non-48k rates	logcat inspection
Automatic USB Hardware	Supported exclusively	Verified via custom USB-IF	System stack locks	DAC front panel
Sample-Rate Switching	via Direct USB Driver	UAC2 driver implementations	rates to 48/96 kHz	sample rate OLED/
without direct app	LED indicator	
Native DSD Bitstreaming	Supported via Direct	Linux ALSA usbfs raw bulk/	System HAL converts	DAC DSD lock LED;
(DSD_U32_BE)	Driver (Hardware-dep.)	isochronous pass-through	all DSD to PCM	USB protocol trace
[span_122](start_span)[span_122](end_span)[span_123](start_span)[span_123](end_span)[span_124](start_span)[span_124](end_span)				
analysis				
DSD over PCM (DoP v1.1)	Supported via Direct	Independent verified tests;	Requires 24/32-bit	DAC DoP decode
	Driver or Framework	DoP 1.1 open standard	PCM bit-perfect	indicator;
	(if unmixed)			
[span_125](start_span)[span_125](end_span)[span_126](start_span)[span_126](end_span)[span_127](start_span)[span_127](end_span)				
delivery	silent noise floor			
Hardware USB DAC Volume	Supported via UAC2	USB Device Class Definition	Host UI must bind	Audio signal level
Control (Mixer Unit)	Control Requests	for Audio Devices v2.0	to UAC2 Volume	without digital
Feature Unit	truncation	

```
|                    |                    |
| Samsung Effects Bypass   | Full Bypass via User- | Architectural
isolation     | App-level DSP must | Bit-null analysis;|
| (Dolby, SoundAlive)      | Space Driver; Partial | from Android
audio framework| be substituted by  | dumpsys media.    |
|                          | via System Direct     |
[span_128](start_span)[span_128](end_span)[span_132](start_span)[span_
132](end_span)            | custom engine      | audio_flinger     |
+--------------------------+----------------------+------------------
----------+--------------------+-------------------+
```

Samsung's One UI 6.1 and 6.1.1 software environments introduce several proprietary processing layers on top of base Android:

- *Dolby Atmos*: Provides specialized profiles for auto, music, movies, voice, and gaming. When active, it inserts dynamic equalizers and phase-widening filters into the system playback chain. These filters remain active for all audio routed through standard framework APIs, modifying the frequency response before it reaches an external USB DAC.
- *Adapt Sound*: Configures multi-band equalization offsets based on hearing tests or age profiles. These equalizations are applied within the system audio server, modifying standard framework streams.
- *SoundAlive and UHQ Upscaler*: Historically, Samsung included an explicit 32-bit "UHQ Upscaler" setting for wired headphones. In recent One UI firmware on the Galaxy S24 series, this setting is disabled or hidden for external USB DACs, and the "UHQ" branding is instead applied to the proprietary Samsung Seamless Codec Ultra High Quality (SSC-UHQ) wireless mode used with the Galaxy Buds3 Pro.
- *USB Audio Framework Routing*: When audio is routed to a USB DAC through standard framework pathways, Samsung's AudioPolicyService directs the stream to the default Android USB Audio HAL. This locks output to a single fixed sample rate (typically 48 kHz or 96 kHz) and mixes in system notifications, preventing bit-perfect transmission.

Technical Architecture of Specialist Android Audio Players

Specialist high-resolution music players on Android achieve bit-perfect output and mixer bypass using distinct driver and framework architectures.

```
+-------------------------------------------------------------------
-----------------------------------------------------+
|                                  SPECIALIST ANDROID MUSIC PLAYER
COMPARATIVE MATRIX                                    |
+--------------------------+----------------------+-----------------
------+---------------------+-----------------------+
| Feature / Subsystem      | USB Audio Player PRO  | Neutron Music
Player   | Poweramp            | HiBy Music (Android)  |
+--------------------------+----------------------+-----------------
------+---------------------+-----------------------+
| Core Driver Model        | Proprietary C User-   | Custom C++
```

```
User-Space  | Dynamic HAL Hook /  | Proprietary User-Space|
|                          | Space UAC1/UAC2 Driver| Driver + Native
Engine | Custom AudioTrack   | USB Driver            |
|                          |                       |
| Direct Flags        |                       |
|                          |                       |
|                          |                       |
| AudioFlinger Bypass      | Complete Bypass       | Complete Bypass
| Partial Bypass      | Complete Bypass       |
| Determinism              | (via usbfs)           | (via usbfs)
| (Depends on OEM HAL)| (via usbfs)           |
|                          |                       |
|                          |                       |
| Sample Rate Switching    | Deterministic via UAC2| Deterministic via
UAC2 | Non-Deterministic   | Deterministic via UAC2|
| Support                  | Clock Control         | Clock Control
| (Subject to OS/HAL) | Clock Control         |
|                          |                       |
|                          |                       |
| Native DSD Support       | Yes                   | Yes
| No (Converts to PCM)| Yes                   |
|                          | (DSD_U32_BE)          | (DSD_U32_BE)
|                      | (DSD_U32_BE)          |
|                          |                       |
|                          |                       |
| DoP v1.1 Framing         | Yes                   | Yes
| Yes (Software pack) | Yes                   |
|                          |                       |
|                          |                       |
| Hardware Volume Control  | Yes (UAC2 Feature     | Yes (UAC2 Feature
| Yes (If supported by| Yes (UAC2 Feature     |
|                          | Unit via Control Ep)  | Unit via Control
Ep)   | HAL direct path)    | Unit via Control Ep)  |
|                          |                       |
|                          |                       |
| Integrated DSP           | Basic 10-band EQ,     | 64-bit
High-Precision  | High-Precision PEQ, | Mage Sound 8-Ball     |
| Subsystems               | Tone, Morphit Profiles| PEQ, Crossfeed,
Delay  | Compressor, Limiter | (MSEB), Parametric EQ |
|                          |                       |
|                          |                       |
| Evidence Classification | Verified (Independent | Verified
(Independent  | Verified (Independent| Verified (Independent  |
|                          | bit-perfect testing)  | bit-perfect
testing)   | bit-perfect testing)| bit-perfect testing)  |
+--------------------------+-----------------------+------------------
------+---------------------+-----------------------+
```

Application Driver Paradigms

USB Audio Player PRO (UAPP, developed by eXtream Software Development) implements a proprietary user-space USB audio driver written in C. When a USB DAC is connected, UAPP requests device permissions via UsbManager, extracts the native Linux file descriptor using UsbDeviceConnection.getFileDescriptor(), and detaches the kernel snd-usb-audio driver. This completely isolates the external DAC from AudioFlinger, the Android audio HAL, and One UI effects . UAPP handles sample-rate switching directly by sending UAC2 Clock Source control requests across Endpoint 0, and manages volume attenuation by addressing the DAC's physical Feature Unit registers.

Neutron Music Player uses a platform-independent 32-bit/64-bit software audio engine. For USB devices, it includes a custom user-space driver operating over Linux usbfs, bypassing system mixers. When communicating with onboard hardware, it instantiates native AudioTrack clients using direct output flags (AUDIO_OUTPUT_FLAG_DIRECT, AUDIO_OUTPUT_FLAG_RAW) to bypass framework processing where supported by the OEM HAL. Neutron provides a 64-bit floating-point DSP pipeline featuring parametric equalization, room acoustic compensation, and crossfeed. When bit-perfect mode is engaged, all DSP stages are bypassed, delivering unaltered PCM or DSD data directly to the driver.

Poweramp uses a dynamic platform output engine. Rather than implementing an independent user-space USB driver, its "Hi-Res Output" relies on undocumented OEM Audio HAL modifications. It configures native AudioTrack instances with vendor-specific parameters, attempting to open a direct, unmixed HAL path at the media's native sampling frequency. While this approach avoids requiring low-level USB device permissions, it is vulnerable to Android updates: if an operating system update alters the underlying audio policy XML or HAL descriptors, the direct path can silently fail, causing the stream to fall back to the 48 kHz system mixer.

HiBy Music functions as both an Android playback application and the core interface for HiBy's digital audio players. On standard smartphones, its "Exclusive HQ USB Audio Access" activates a user-space driver for connected USB DACs. On its proprietary DAPs, HiBy replaces Android's standard framework with its Direct Transport Audio (DTA) architecture, which bypasses AudioFlinger's 48 kHz resampler globally across all applications. HiBy Music also incorporates Mage Sound 8-Ball (MSEB), an intuitive equalization system that maps subjective acoustic qualities directly to parametric biquad filter adjustments.

Bit-Perfect Audio Integrity and Operating System Bypass Mechanisms

Mathematical Definition of Bit-Perfect Delivery

A digital playback pathway is mathematically bit-perfect if the sequence of N-bit discrete samples x[n] delivered to the input of the DAC's digital filter matches the sequence of discrete samples s[n] extracted from the decoded container:

$$x[n] = s[n], \quad \forall n \in [0, M-1]$$

where M is the total length of the audio sequence in frames, and physical transmission occurs at

the original source sampling frequency:
f_{\text{target}} = f_{\text{source}}
Bit-perfect delivery preserves:

- Sample parity: No samples are dropped, duplicated, or interpolated.
- Bit depth fidelity: Quantization levels are preserved without truncation or requantization dither.
- Gain neutrality: Audio data remains at absolute unity gain (0\text{ dBFS}) prior to hardware conversion.
- Spectral accuracy: The Nyquist bandwidth is maintained without passband ripple or phase shifts from software filters.

Any digital modification—including a 0.1\text{ dB} volume reduction, ReplayGain normalization, software equalization, or sample-rate conversion from 44.1 kHz to 48 kHz—breaks bit-for-bit equality. While these DSP features may be intentionally selected to tailor acoustic output, they do not constitute bit-perfect delivery.

Resampling Mechanics and Android 14+ AudioMixerAttributes

In standard Android operation, when an application passes a 44.1 kHz stream to AudioTrack, AudioFlinger determines whether an existing hardware output thread matches that rate. Because mobile device output mixers are generally locked to 48 kHz to maintain low latency for telephony and system UI sounds, the stream is routed to AudioMixer's resampling algorithm.
To address mixer resampling, Android 14 (API level 34) introduced an expanded mixer configuration API:

```
// Discover supported mixer configurations for an external USB DAC
AudioDeviceInfo usbDevice = ...; // Obtained via
AudioManager.getDevices()
List<AudioMixerAttributes> mixerAttrs =
audioManager.getSupportedMixerAttributes(usbDevice);

// Select a bit-perfect configuration matching source media (e.g., 96
kHz, 24-bit stereo)
AudioFormat targetFormat = new AudioFormat.Builder()
        .setSampleRate(96000)
        .setChannelMask(AudioFormat.CHANNEL_OUT_STEREO)
        .setEncoding(AudioFormat.ENCODING_PCM_24BIT_PACKED)
        .build();

AudioMixerAttributes bitPerfectAttrs = new
AudioMixerAttributes.Builder(targetFormat)

.setMixerBehavior(AudioMixerAttributes.MIXER_BEHAVIOR_BIT_PERFECT)
        .build();

// Set preferred mixer attributes to bypass AudioFlinger processing
boolean status =
audioManager.setPreferredMixerAttributes(targetFormat, usbDevice,
bitPerfectAttrs);
```

When successfully applied, MIXER_BEHAVIOR_BIT_PERFECT instructs AudioFlinger to route the stream directly to the underlying Audio HAL without resampling, floating-point expansion, or system effects.
However, real-world deployment on commercial devices running Android 14 and 15 reveals notable limitations:

1. *Vendor HAL Compatibility*: The underlying vendor Audio HAL must implement the corresponding AIDL/HIDL interfaces correctly. On many production devices—including several Samsung and Google Pixel models—calling setPreferredMixerAttributes() can fail, throw runtime exceptions, or cause audio daemon crashes (aocd or audioserver) when requesting non-standard sample rates such as 88.2 kHz, 176.4 kHz, or 352.8 kHz.
2. *Lifecycle Contention*: If a higher-priority audio event occurs (such as an incoming cellular call), the framework bit-perfect stream is forcibly torn down or muted, which can trigger loud digital transients or driver stalls.
3. *Lack of DSD Support*: Android framework APIs only recognize PCM encodings. Native DSD bitstreaming cannot be routed through AudioMixerAttributes without wrapping the stream in PCM frames using DSD over PCM (DoP).

Consequently, while AudioMixerAttributes.MIXER_BEHAVIOR_BIT_PERFECT provides a useful framework-level mechanism, a dedicated user-space USB Audio Class driver remains the most deterministic approach for reliable, bit-perfect output across all sample rates and formats.

High-Resolution Digital Audio Reproduction Mechanics

Evaluating the merits of high-resolution digital playback requires examining signal processing theory, human auditory limits, and the physical behavior of digital-to-analog converters.

Sampling Theory and Ultrasonic Reconstruction

The Nyquist-Shannon sampling theorem states that any continuous band-limited analog signal containing no frequency components higher than B can be perfectly captured and reconstructed from discrete samples taken at a uniform rate f_s, provided:
f_s > 2B
For human hearing, the nominal upper limit of auditory perception is approximately 20\text{ kHz}. A sampling rate of 44.1\text{ kHz} captures frequencies up to 22.05\text{ kHz}, providing complete coverage of the audible spectrum.
High-resolution sampling rates (such as 96 kHz or 192 kHz) do not expand the audible frequency spectrum. However, they offer practical engineering advantages in digital-to-analog converter design:

- *Filter Transition Band Relief*: At f_s = 44.1\text{ kHz}, the transition band between the passband edge (20\text{ kHz}) and the stopband edge (22.05\text{ kHz}) is only 2.05\text{ kHz} wide. Reconstructing this signal requires a steep anti-aliasing or anti-imaging "brickwall" filter. Steep linear-phase filters introduce pre-ringing and post-ringing in the time domain, while minimum-phase filters introduce phase non-linearities in the high-frequency band. At f_s = 96\text{ kHz} or 192\text{ kHz}, the transition band expands considerably (e.g., from 20\text{ kHz} to 48\text{ kHz}), allowing DAC designers to use gentle, slow-roll-off reconstruction filters that minimize time-domain ringing and phase shifts within the audible band.

- *Ultrasonic Intermodulation Distortion (IMD)*: Wide-bandwidth audio reproduction also introduces potential engineering challenges. If high-resolution recordings contain high-amplitude ultrasonic noise (often caused by aggressive noise shaping in delta-sigma modulators or high-bandwidth microphone preamps), these frequencies can enter downstream analog amplifiers and non-linear headphone drivers. This can generate audible intermodulation distortion products:

f_{\text{intermod}} = \vert{}f_1 - f_2\vert{}

These intermodulation products can fold back into the audible spectrum, creating false acoustic artifacts that degrade sound quality.

Dynamic Range and Quantization Accuracy

Quantization depth determines the dynamic range and noise floor of a digital audio system. The theoretical signal-to-quantization-noise ratio (SQNR) for an ideal un-dithered N-bit system is given by:

\text{SQNR} \approx 6.02 \cdot N + 1.76\text{ dB}

Applying this formula highlights clear differences in dynamic range:

- *16-bit Quantization*: Delivers an SQNR of approximately 98.1\text{ dB}. With proper TPDF dither, a 16-bit system can resolve signals well below the least significant bit, reaching an effective dynamic range of over 115\text{ dB}, which exceeds the ambient acoustic limits of most listening environments.
- *24-bit Quantization*: Yields a theoretical dynamic range of 146.2\text{ dB}. Modern high-end DACs achieve real-world dynamic ranges between 120\text{ dB} and 132\text{ dB}, with performance limited by the thermal noise (Johnson-Nyquist noise) of analog resistors and operational amplifiers. 24-bit resolution ensures that digital transmission remains well below the analog noise floor of the physical hardware, preserving low-level detail without requantization noise during software volume adjustments.
- *32-bit Integer Processing*: Provides up to 194.4\text{ dB} of dynamic range, which far exceeds physical reproduction capabilities. Its primary purpose is serving as an intermediate container to preserve precision during complex DSP pipelines and avoid round-off error accumulation.

USB-C Audio Architecture and External DAC Integration

USB Audio Class Standards and Clock Synchronization

The Universal Serial Bus Implementers Forum (USB-IF) defines two primary standards for transferring audio across USB links:

```
+-------------------------------------------------------------------------
-----------------------------------------------+
|                                    USB AUDIO CLASS ARCHITECTURAL
COMPARISON                                      |
+--------------------------+-----------------------------------------
---+-----------------------------------------------+
| Architectural Property   | USB Audio Class 1.0 (UAC1)
```

```
| USB Audio Class 2.0 (UAC2)                        |
+--------------------------+-----------------------------------------
---+-------------------------------------------------+
| USB Bus Speed            | Full Speed (12 Mbps)
| High Speed (480 Mbps)                              |
| Isochronous Packet Limit | 1023 bytes per 1 ms frame
| 1024 bytes per 125 microframe (8x/ms)              |
| Maximum Bandwidth        | ~8.192 Mbps (Isochronous limit)
| ~196.6 Mbps (Isochronous limit)                    |
| Maximum Format Envelope  | 2-channel 24-bit/96 kHz
| Multi-channel 32-bit/768 kHz, Native DSD1024       |
| Clock Synchronization    | Synchronous, Adaptive, or Asynchronous
| Asynchronous (Mandatory explicit feedback)         |
| Driver Requirement       | Universal native operating system support
| Native support on modern Linux/Android/Windows     |
| Control Mechanism        | UAC1 Feature Units, Clock Controls
| UAC2 Clock Source, Selector, and Feature Units     |
+--------------------------+-----------------------------------------
---+-------------------------------------------------+
```

High-resolution, bit-perfect playback depends on UAC2's asynchronous transmission mode. In this mode, the external USB DAC acts as the clock master, relying on its internal, temperature-compensated crystal oscillators to set the sample timing.
Because the host phone's internal clock runs on a separate timebase, the DAC continuously monitors its internal FIFO buffer and transmits flow-control updates back to the host using an explicit feedback isochronous endpoint. The feedback value:
F_f = \frac{f_{\text{DAC}}}{f_{\text{bus}}}
instructs the host driver to dynamically modulate the number of PCM samples it schedules per microframe. This mechanism prevents buffer overruns and underruns while avoiding digital clock jitter on the audio lines.

Android USB Host Limitations: The Isochronous Hurdle

The standard Android Java framework includes host communication APIs under android.hardware.usb (UsbManager, UsbDeviceConnection, UsbRequest, and U[span_187](start_span)[span_187](end_span)[span_188](start_span)[span_188](end_span)[span_189](start_span)[span_189](end_span)sbEndpoint).
However, UsbRequest only supports Bulk and Interrupt transfer types. Although UsbConstants.USB_ENDPOINT_XFER_ISOC is defined in the Android SDK, calling UsbRequest.initialize() on an isochronous endpoint fails or throws an exception on standard production builds.
Because real-time USB audio data is transferred using isochronous endpoints—which reserve dedicated bus bandwidth without transmission retries—applications cannot implement a high-resolution UAC2 audio driver purely in Java or Kotlin.

Implementing a Native User-Space USB Audio Driver via usbfs

To build an application-controlled USB audio engine, a developer must bridge Android's

high-level permissions system with low-level Linux USB device nodes.
The application discovers connected peripherals using UsbManager.getDeviceList(), presents an attachment filter via an XML resource profile, and prompts the user for explicit USB permissions. Once granted, the application opens a UsbDeviceConnection and extracts the native integer file descriptor using UsbDeviceConnection.getFileD[span_199](start_span)[span_199](end_span)[span_200](start_span)[span_200](end_span)escriptor().
This file descriptor is passed across the JNI bridge to the native C++ engine. In user space, the engine claims the streaming interface by issuing the USBDEVFS_DISCONNECT_CLAIM ioctl call to the Linux USB filesystem (usbfs), which detaches the kernel snd-usb-a[span_201](start_span)[span_201](end_span)[span_202](start_span)[span_202](end_span)udio driver. The application then switches the interface from Alternate Setting 0 (zero bandwidth) to Alternate Setting 1 (active isochronous streaming) using USBDEVFS_SETINTERFACE.
To set the hardware clock, the engine dispatches a UAC2 SET_CUR control request across Endpoint 0 targeting the terminal Clock Source ID. Finally, the driver enqueues double-buffered asynchronous isochronous transfer requests using the USBDEVFS_SUBMITURB ioctl and harvests completed frames via USBDE[span_203](start_span)[span_203](end_span)[span_204](start_span)[span_204](end_span)[span_205](start_span)[span_205](end_span)VFS_REAPURB.

```
// Native implementation: Submitting an asynchronous isochronous URB
via usbfs
#include <linux/usbdevice_fs.h>
#include <sys/ioctl.h>
#include <cstring>

bool
submit_isochronous_audio_frame(int[span_92](start_span)[span_92](end_s
pan) fd, unsigned char endpoint_addr,
                                    unsigned char* pcm_data, int
packet_size,
                                    int num_packets) {
    struct usbdevfs_urb urb;
    std::memset(&urb, 0, sizeof(urb));

    urb.type = USBDEVFS_URB_TYPE_ISO;
    urb.endpoint = endpoint_addr;
    urb.status = 0;
    urb.flags = USBDEVFS_URB_ISO_ASAP;
    urb.buffer = pcm_data;
    urb.buffer_length = packet_size * num_packets;
    urb.number_of_packets = num_packets;

    struct usbdevfs_iso_packet_desc iso_descriptors[num_packets];
    for (int i = 0; i < num_packets; ++i) {
        iso_descriptors[i].length = packet_size;
        iso_descriptors[i].actual_length = 0;
        iso_descriptors[i].status = 0;
```

```
    }
    urb.iso_frame_desc = iso_descriptors;

    if (ioctl(fd, USBDEVFS_SUBMITURB, &urb) < 0) {
        return false;
    }
    return true;
}
```

This user-space approach bypasses AudioFlinger, the Android audio HAL, and Samsung One UI system post-processing . The application controls clock frequency selection directly via UAC2 control transfers, enabling automatic sample-rate switching across all supported frequencies. It also enables native DSD bitstreaming and low-jitter asynchronous isochronous streaming that the standard Android HAL does not support.

DSD Streaming Architectures: Native Bitstreaming vs. DSD over PCM (DoP)

Direct Stream Digital (DSD) stores audio using single-bit delta-sigma modulation at high oversampling multiples of 44.1 kHz: DSD64 (2.8224 MHz), DSD128 (5.6448 MHz), and DSD256 (11.2896 MHz). Delivering DSD across USB to an external DAC can be accomplished using two distinct methods:
In Native DSD Transmission (DSD_U32_BE / raw encapsulation), raw single-bit data is packed into 32-bit words and transmitted directly over standard isochronous endpoints. This requires the external DAC to implement a vendor-specific UAC2 alternate interface capable of receiving raw single-bit streams. Because Linux ALSA historically required custom patch sets for each DAC model's USB Vendor/Product ID to enable native DSD, an Android user-space driver must maintain its own internal table of supported DAC IDs to correctly initialize raw DSD mode.

DoP v1.1 24-BIT FRAME STRUCTURE		
Bit Range	Field Name	Description
Bits 23 to 16	Synchronization Header Marker	Alternates between 0x05 (odd) and 0xFA (even frames)
Bits 15 to 0	Raw DSD Audio Payload	16 consecutive 1-bit DSD audio samples

To avoid requiring custom vendor drivers, the DSD over PCM (DoP v1.1) open standard packages raw DSD bitstreams inside standard 24-bit PCM audio frames. Because DSD64

requires a bit rate of 2.8224\text{ Mbps} per channel, mapping it into a stereo 16-bit PCM stream requires a transmission rate of 176.4\text{ kHz} (176.4\text{ kHz} \times 16\text{ bits} = 2.8224\t[span_224](start_span)[span_224](end_span)[span_226](start_span)[span_226](end_span)ext{ Mbps}).
The DoP standard packages 16 bits of raw DSD data into the lower payload bits of a 24-bit PCM frame, and uses the upper 8 bits to carry alternating synchronization markers: 0x05 on odd frames and 0xFA on even frames. When a DoP-compatible DAC detects this alternating marker pattern across 32 consecutive frames, it switches its internal signal path to DSD mode without generating digital switching noise.
Crucially, because DoP travels inside standard 24-bit PCM frames, **the digital transport must remain completely bit-perfect**. Any digital volume scaling, sample-rate conversion, or mixing will modify the marker bytes. If the marker sequence is corrupted, the DAC falls back to standard PCM decoding, causing the raw DSD bitstream to be converted as loud, white quantization noise.

Bluetooth Audio Subsystem and Wireless Constraints

Audio routed to wireless headphones passes through an entirely different technical pipeline than wired USB audio. The decoded stream passes through AudioFlinger, where it is resampled to the rate negotiated by the Bluetooth Audio HAL (typically 48 kHz or 96 kHz) and subjected to system equalizers and volume attenuation curves. The stream then enters the Bluetooth daemon (Fluoride/Gabeldorsche), which feeds an active codec encoder. The encoded data is packetized into A2DP transport frames and routed over the 2.4 GHz radio link via the HCI baseband interface to the receiving headphones' onboard DAC.
A third-party Android application running without root permissions cannot bypass the system Bluetooth encoder. The Android Bluetooth architecture exposes no public APIs that permit arbitrary user-space binaries to replace the system A2DP codec libraries or inject raw encoded packets directly into the Host Controller Interface (HCI). The application must write PCM samples to AudioTrack, leaving sample rate conversion, codec selection, dynamic bitrate scaling, and radio packetization entirely under the control of AudioFlinger, BluetoothAudioHAL, and the system Bluetooth daemon.

BLUETOOTH AUDIO CODEC COMPARATIVE PROFILE			
Codec Identifier	Max Bandwidth	Max Sample Rate / Bit Depth	Compression Type & Limitations
SBC	328 kbps (Joint Stereo)	44.1 / 48 kHz 16-bit	Lossy Mandatory baseline; coarse psycho-

```
(Subband ADPCM)     | acoustic masking; audible artifacts  |
|                   |                 |                      |
|                                       |
| AAC               | ~256 to 320     | 44.1 / 48 kHz        | Lossy
| Advanced perceptual audio coding;    |
|                   | kbps (VBR)      | 16-bit / 24-bit      | (MDCT
transform)    | Android encoder exhibits variable    |
|                   |                 |                      |
| psychoacoustic implementation quality|
|                   |                 |                      |
|                                       |
| aptX / aptX HD    | 352 kbps /      | 48 kHz               | Lossy
| Time-domain ADPCM with fixed 4:1     |
|                   | 576 kbps        | 16-bit / 24-bit      |
(Subband ADPCM)     | compression ratio; preserves         |
|                   |                 |                      |
| transients; limited dynamic headroom |
|                   |                 |                      |
|                                       |
| LDAC              | 330 / 660 /     | 96 kHz               | Lossy
| High bitrates (990 kbps) cover up to |
|                   | 990 kbps        | 24-bit               |
(Modified MDCT)     | 40 kHz; dynamic link throttling drops|
|                   |                 |                      |
| bandwidth to 330 kbps in RF noise    |
|                   |                 |                      |
|                                       |
| LC3 (LE Audio)    | Up to 392 kbps  | 48 kHz               | Lossy
| Successor to SBC; higher perceptual  |
|                   | per channel     | 24-bit / 32-bit      | (Low
Complexity MDCT| quality per bit; low transport       |
|                   |                 |                      |
subband)            | latency                          |
|                   |                 |                      |
|                                       |
| Samsung Seamless  | Proprietary     | 96 kHz               | Dynamic
Lossy       | Samsung ecosystem exclusive;         |
| Codec (SSC-UHQ)   | adaptive (~512+ | 24-bit               |
(Proprietary subband| available on Galaxy S24+ with        |
|                   | kbps)           |                      |
encoding)           | Buds3 Pro; adaptive bitrate scaling  |
+-------------------+-----------------+----------------------+--------
------------+--------------------------------------+
```

Claims of bit-perfect wireless playback over standard Bluetooth A2DP connections are fundamentally inaccurate. SBC, AAC, aptX, and LDAC all discard spectral data using psychoacoustic masking models to fit within the bandwidth constraints of Bluetooth radio links. Furthermore, although LDAC can transmit 24-bit/96 kHz audio containers at 990 kbps, standard

44.1 kHz Red Book CD audio is typically upsampled to 96 kHz by the operating system before encoding, modifying the original bitstream. In environments with high 2.4 GHz radio-frequency interference, LDAC automatically steps down from 990 kbps to 660 kbps or 330 kbps to maintain connection stability, increasing compression loss.
While Qualcomm has introduced an "aptX Lossless" mode within its Snapdragon Sound suite, this feature requires proprietary Qualcomm RF hardware and is not supported on the Exynos 2400 platform. Consequently, an Android music player engine must treat Bluetooth output as a lossy playback path. The engine should focus on maximizing transmission headroom, preventing digital clipping, and avoiding unnecessary software sample-rate conversions prior to encoding.

Dedicated Digital Audio Player (DAP) Hardware and Software Engineering

High-end Digital Audio Players from manufacturers such as Sony, FiiO, Astell&Kern, and HiBy employ tailored hardware and software architectures to overcome the technical limitations of consumer smartphones.

```
+-------------------------------------------------------------------------
-------------------------------------------------+
|               [span_52](start_span)[span_52](end_span)
DAP TECHNOLOGY REPRODUCIBILITY MATRIX
|
+--------------------------+-----------------------+------------------
----------+--------------------+--------------------+
| Technology / Feature     | Manufacturer          | Underlying
Implementation   | App-Only           | Hardware / System |
|
[span_55](start_span)[span_55](end_span)[span_58](start_span)[span_58]
(end_span)      |                         | Mechanism
| Implementation?    | Privilege Req.?   |
+--------------------------+-----------------------+------------------
----------+--------------------+--------------------+
| Global System SRC Bypass | HiBy (DTA) /          | Modified AOSP
frameworks    | No                 | Requires System   |
| Across All Android Apps  | FiiO (DAPS)           | and Audio HAL
libraries     |                    | Platform Signatures|
|                          |
[span_240](start_span)[span_240](end_span)[span_241](start_span)[span_
241](end_span)[span_242](start_span)[span_242](end_span)[span_243](sta
rt_span)[span_243](end_span)          |       |                    |
or Custom ROM     |
|                          |                       |
|                     |                    |
| Ultra-Low Phase Noise    | Sony / FiiO /         | Dual dedicated
crystal      | No                 | Dedicated Physical|
| Hardware Clock Domains   | Astell&Kern           | oscillators +
FPGA routing  |                    | Silicon Hardware  |
```

Balanced High-Voltage Headphone Amplification	Sony / Astell&Kern / FiiO	Multi-rail analog circuits, discrete op-amps / buffers	No	Dedicated Physical Silicon Hardware
AI High-Frequency Reconstruction (Upscale)	Sony (DSEE Ultimate) (Custom DSP model)	Machine-learning spectral prediction algorithms	Yes	High CPU overhead; App-level software
Analog Phase Profile Emulation	Sony (DC Phase Linearizer) (Biquad DSP engine)	Low-frequency allpass / IIR phase-shifting filters	Yes	App-level software (Mathematically achievable)
Acoustic Turntable Modeling	Sony (Vinyl Processor) (Non-linear DSP)	Non-linear harmonics, phase shift, and rumble DSP	Yes	App-level software (Mathematically achievable)

Real-Time PCM to	Sony (DSD Remaster) /	High-order multi-stage	Partially	Extremely high CPU
DSD256 Conversion	FiiO	Delta-Sigma Modulator	(Performance-	consumption; heavy
	[span_280](start_span)[span_280](end_span)[span_283](start_span)[span_283](end_span)[span_286](start_span)[span_286](end_span)	
constrained)	battery drain

Hardware and Operating System Architectures

Smartphones derive clock frequencies for the CPU, display, baseband, and audio busses from a single system crystal oscillator using Fractional Phase-Locked Loops (PLLs), which can introduce phase jitter into digital audio lines. In contrast, dedicated DAPs incorporate two independent, low-phase-noise crystal oscillators (e.g., NDK femtosecond crystals): one dedicated to multiples of 44.1 kHz (45.1584 MHz) and one dedicated to multiples of 48 kHz (49.1520 MHz) . An onboard FPGA acts as a clock distribution processor, routing jitter-free I2S audio directly to the DAC.

To prevent Android from forcing all audio through its 48 kHz system mixer, DAP manufacturers customize the core operating system. HiBy's Direct Transport Audio (DTA) and FiiO's customized Android framework modify AudioPolicyService and Audio HAL source code directly. This system-level modification bypasses the AudioFlinger resampler globally, enabling bit-perfect audio delivery across all installed third-party applications (such as Tidal, Apple Music, and Qobuz) to internal DACs. Modern DAPs also incorporate high-capacity linear power regulators, film capacitors, and fully balanced 4.4mm Pentaconn analog output stages capable of delivering continuous power (often exceeding 500\text{--}1500\text{ mW} into [span_296](start_span)[span_296](end_span)[span_297](start_span)[span_297](end_span)[span_298](start_span)[span_298](end_span)[span_299](start_span)[span_299](end_span)[span_300](start_span)[span_300](end_span)32\,\Omega).

Sony Proprietary Signal Processing Technologies

Sony DAPs (such as the NW-WM1ZM2 and NW-ZX707) include specialized DSP features implemented either in FPGA/DSP silicon or within native audio framework daemons:

- *DSEE Ultimate (Digital Sound Enhancement Engine)*: A real-time software/DSP upscaling technology. Using machine learning algorithms trained on high-resolution acoustic models, DSEE analyzes compressed or CD-standard PCM audio to predict and reconstruct high-frequency harmonics cut off by compression or steep anti-aliasing filters, upsampling streams in real time up to 32-bit/192 kHz .
- *DC Phase Linearizer*: Traditional analog amplifiers use output-coupling capacitors and feedback loops that introduce phase shifts in the lowest octaves (below 100[span_308](start_span)[span_308](end_span)[span_309](start_span)[span_309](end_span)\text{ Hz}). Sony's proprietary S-Master HX digital amplifier operates as a switching power converter without analog phase shift, producing a flat low-frequency phase curve. The DC Phase Linearizer applies specialized IIR filtering to reproduce the low-frequency

phase curves of traditional analog amplifiers, offering six distinct phase profiles (labeled Type A/B: low, standard, high) .
- *Vinyl Processor*: A DSP feature designed to simulate the analog characteristics of vinyl record playback . It introduces controlled sub-audible low-frequency rumble (mimicking tonearm resonance between 10 Hz and 15 Hz), subtle low-order harmonic warmth, high-frequency phase dispersion, and turntable motor acoustic feedback modeling .
- *DSD Remastering Engine*: Uses a dedicated DSP or FPGA to convert all incoming PCM bitstreams into an 11.2 MHz single-bit DSD stream (DSD256) via real-time delta-sigma modulation, allowing the stream to be converted using a gentle analog low-pass reconstruction filter .

An application developer targeting the Galaxy S24+ cannot reproduce hardware-bound features, such as low-jitter physical crystal oscillators or balanced analog amplification stages, in software alone. These benefits are obtained by connecting an asynchronous external USB DAC. However, acoustic modeling algorithms—such as phase linearizers, vinyl acoustic emulation, parametric tone adjustments, and neural-network-assisted spectral upscaling—are fundamentally digital signal processing techniques that can be implemented directly within a custom native DSP engine.

Advanced Digital Signal Processing (DSP) Pipeline Design

A modern high-resolution audio engine must balance two core operational priorities: providing bit-perfect hardware bypass when pure data fidelity is required, and offering high-precision DSP when users wish to calibrate their acoustic environment.

The DSP pipeline operates on 64-bit double-precision floating-point values to preserve dynamic range and numerical stability during multi-stage processing. When active, the pipeline processes audio through the following stages:

1. *Format Promotion*: Incoming integer or 32-bit float PCM samples are converted to IEEE 754 64-bit double-precision floats.
2. *Loudness Normalization*: ReplayGain 2.0 / EBU R128 processing applies track or album gain offsets alongside an oversampled true-peak limiter.
3. *Parametric Equalization*: Cascaded Direct Form II Transposed biquad filter arrays adjust frequency response with automated pre-attenuation gain calculation.
4. *Partitioned FIR Convolution*: Uniformly partitioned overlap-save FFT convolution applies headphone correction profiles (e.g., AutoEq) or room impulse responses.
5. *Acoustic Crossfeed*: A Linkwitz-Bauer modeled delay-and-shelf network reduces headphone listening fatigue.
6. *Polyphase Resampling*: Band-limited sinc resampling (via libsoxr) adjusts sample rates when explicitly requested.
7. *True-Peak Limiting*: An oversampled soft-knee limiter prevents inter-sample clipping.
8. *TPDF Dithering and Quantization*: High-resolution Triangular Probability Density Function dither is applied before re-quantizing the stream to the target hardware bit depth (16-bit, 24-bit, or 32-bit integer).

Parametric Equalizer Design

Parametric equalization modifies frequency response without the phase distortion and band

interference typical of basic graphic equalizers. The engine implements each filter band using a Direct Form II Transposed biquadratic filter structure, which improves numerical stability and reduces quantization noise during floating-point operations. The continuous transfer function in the z-domain is expressed as:
$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{a_0 + a_1 z^{-1} + a_2 z^{-2}}$$
Normalized coefficients ($a_0 = 1$) are calculated using Robert Bristow-Johnson's Audio EQ Cookbook formulas. For a parametric peaking filter operating at center frequency f_0, sample rate f_s, gain $A = 10^{\frac{\text{dBGain}}{40}}$, and quality factor Q:
$$\omega_0 = 2\pi \frac{f_0}{f_s}, \quad \alpha = \frac{\sin(\omega_0)}{2Q}$$
The biquad coefficients are derived as:
$$b_0 = 1 + \alpha A, \quad b_1 = -2\cos(\omega_0), \quad b_2 = 1 - \alpha A \quad a_0 = 1 + \frac{\alpha}{A}, \quad a_1 = -2\cos(\omega_0), \quad a_2 = 1 - \frac{\alpha}{A}$$
To prevent digital clipping when boosting filter gains, the engine calculates the maximum positive excursion across all active filter bands:
$$G_{\text{peak}} = \max_{\omega \in [0, \pi]} \left\vert{} \prod_{k=1}^{K} H_k(e^{j\omega}) \right\vert{}$$
and automatically applies an equivalent pre-attenuation gain offset:

$$\text{Gain}_{\text{pre}} = -\max(0\text{ dB}, G_{\text{peak}})$$

Partitioned Convolution Engine

Direct time-domain convolution of an input signal $x[n]$ with an impulse response $h[n]$ of length L:
$$y[n] = \sum_{k=0}^{L-1} x[n-k] h[k]$$
requires $\mathcal{O}(L)$ operations per sample, leading to excessive CPU utilization when handling long impulse responses (such as $L = 131{,}072$ samples for high-resolution acoustic profiles).
To minimize CPU overhead and latency, the engine implements uniformly partitioned overlap-save FFT convolution. The impulse response $h[n]$ is segmented into P smaller blocks of length B. Fast Fourier Transforms are computed on each block once during initialization. As incoming audio blocks arrive, the engine computes their short-length FFTs, performs complex multiplication against the stored impulse response spectra, and accumulates the results in the frequency domain before computing an Inverse FFT. This optimization reduces computational complexity to:
$$\mathcal{O}\left(P \cdot \log_2(2B)\right)$$
This allows high-resolution FIR profiles to run efficiently on mobile CPU cores.

Acoustic Crossfeed Calibration

Headphone listening isolates the left and right audio channels completely to their respective ears, creating an unnatural lateral acoustic image that can cause listening fatigue. In a natural acoustic space, sounds from a source on the left arrive at the left ear first, and reach the right ear with a short time delay and modified frequency response due to head shadowing.
The engine implements acoustic crossfeed using a Linkwitz-Bauer modeled delay-and-shelf network:
$$y_L[n] = x_L[n] + \mathcal{F}^{-1}\left\{ X_R(e^{j\omega}) \cdot H_{\text{shelf}}(e^{j\omega}) \right\}[n - \tau] \quad y_R[n] = x_R[n] + \mathcal{F}^{-1}\left\{ X_L(e^{j\omega}) \cdot H_{\text{shelf}}(e^{j\omega}) \right\}[n - \tau]$$
where $\tau \approx 250\text{--}300\,\mu\text{s}$ represents the interaural time delay (ITD), and

H_{\text{shelf}} is a low-pass shelf filter providing 6\text{--}8\text{ dB} of attenuation above 700\text{ Hz} to simulate head-shadowing interaural level delay (ILD).

ReplayGain 2.0 and True-Peak Normalization

To eliminate abrupt volume differences across tracks and albums, the engine implements loudness normalization based on ITU-R BS.1770 / EBU R128 standards. The engine reads embedded ReplayGain metadata tags (REPLAYGAIN_TRACK_GAIN, REPLAYGAIN_ALBUM_GAIN, and REPLAYGAIN_TRACK_PEAK). The linear target gain is calculated as:
\text{Gain}_{\text{linear}} = 10^{\frac{\text{TargetGain} + \text{PreAmp}}{20}}
If applying this gain boost could cause sample values to exceed 0\text{ dBFS} (1.0 in floating-point representation), the engine activates an oversampled true-peak limiter. The limiter uses a 4\times oversampling polyphase interpolation filter to detect inter-sample peaks that standard peak meters miss. If an inter-sample peak exceeds -0.2\text{ dBFS}, the limiter adjusts the gain smoothly using an attack-release envelope, preventing downstream DAC reconstruction stages from clipping or generating transient distortion.

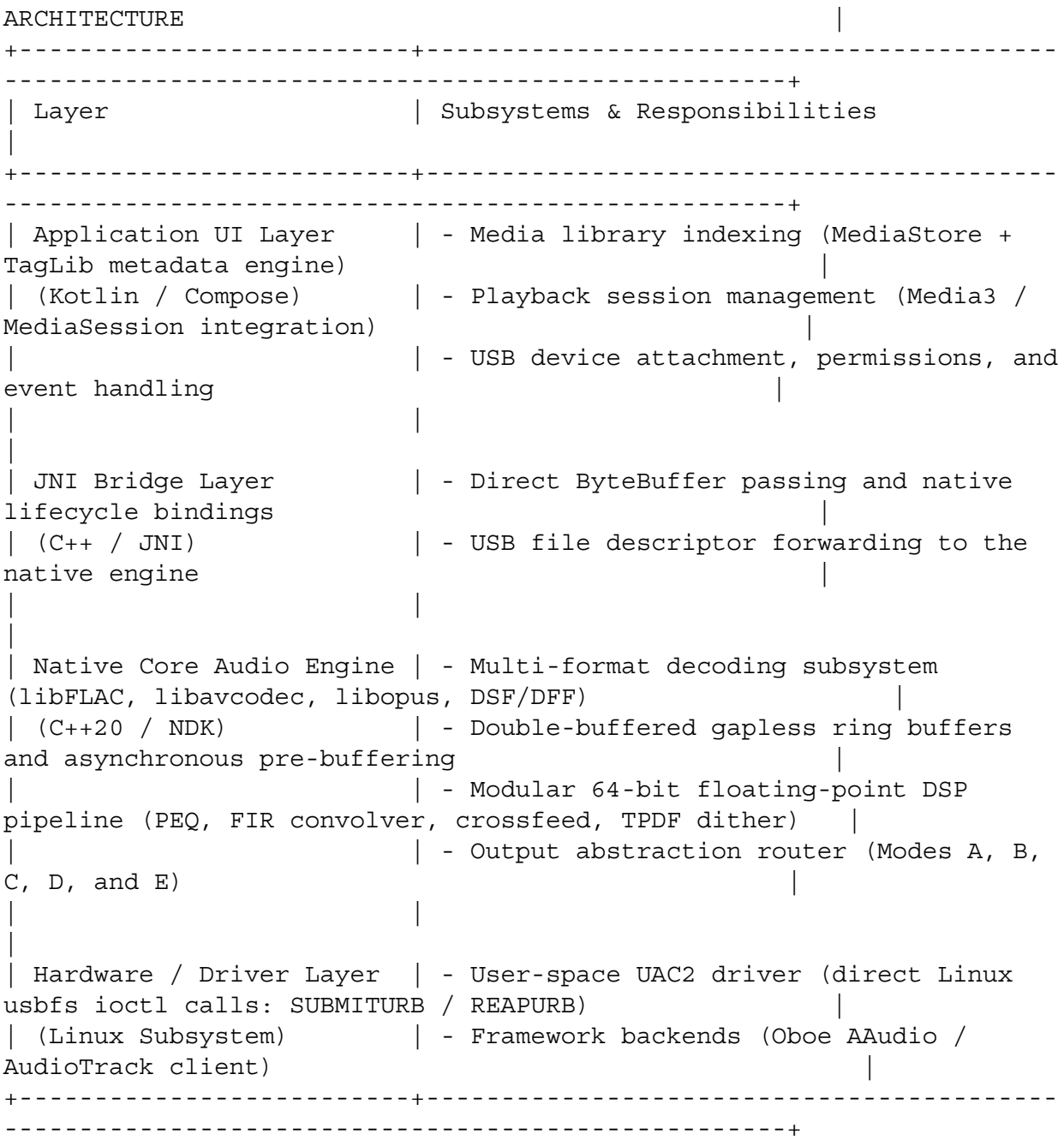
High-Precision Volume Management and TPDF Dithering

Software volume reduction multiplies sample values by an attenuation coefficient:
\alpha = 10^{\frac{\text{dBVolume}}{20}}, \quad \alpha \le 1.0
When audio is processed and stored in 64-bit floating-point format, dynamic range exceeds 300\text{ dB}, preserving low-level detail without audible degradation during mathematical processing. However, when re-quantizing that floating-point stream back to integer format (e.g., 24-bit or 16-bit) for physical transmission to a DAC, simple rounding or bit truncation can introduce quantization errors correlated with the signal, producing harmonic distortion.
To linearize the quantization process, the engine generates two statistically independent random variables uniformly distributed over the interval [-0.5, +0.5] LSB:
R_1, R_2 \sim \mathcal{U}(-0.5, +0.5)
Summing these variables produces Triangular Probability Density Function (TPDF) dither noise:
D_{\text{tpdf}} = R_1 + R_2
The dither noise is added to the scaled sample before integer truncation:
x_{\text{quantized}}[n] = \left\lfloor x_{\text{float}}[n] \cdot 2^{N-1} + D_{\text{tpdf}} + 0.5 \right\rfloor
This replaces signal-correlated distortion harmonics with a continuous, statistically benign white noise floor. When connecting to external USB DACs that support hardware volume control, the engine uses the DAC's onboard attenuator via UAC2 control requests, keeping the software audio stream unattenuated at 0\text{ dBFS} to preserve maximum signal resolution.

Proposed Audio Engine Software Architecture

The proposed high-resolution player uses a modular, layered architecture built in C++20 via the Android NDK, with an interface layer written in Kotlin.

```
+-------------------------------------------------------------------
-------------------------------------------------+
|                                        PROPOSED AUDIO ENGINE LAYERED
```



Discrete Operational Output Modes

The engine provides five distinct output modes to support different hardware configurations and listening preferences:

- *Mode A (Standard Android Compatible)*: Uses AAudio (Shared Mode) or AudioTrack to output audio through AudioFlinger. This mode provides universal compatibility across internal speakers, wired adapters, and Android Auto, while allowing system notifications to mix into the playback stream.
- *Mode B (Android 14+ Bit-Perfect Passthrough)*: Uses AudioTrack configured with

AudioMixerAttributes.MIXER_BEHAVIOR_BIT_PERFECT via AudioManager.setPreferredMixerAttributes(). This mode provides framework-level mixer bypass when the user prefers not to grant direct USB device permissions, falling back gracefully to Mode A if the HAL rejects the requested sample rate.

- *Mode C (Dedicated User-Space USB Audio - Bit-Perfect)*: Uses the custom native UAC2 driver operating over usbfs to deliver bit-perfect audio to external USB DACs. This mode completely bypasses AudioFlinger, the system Audio HAL, and One UI effects . Incompatible DSP features are automatically disabled, while automatic sample-rate switching, native DSD, and DoP v1.1 are enabled.
- *Mode D (Dedicated User-Space USB Audio - Studio DSP)*: Combines the custom UAC2 driver with the internal 64-bit floating-point DSP pipeline. This mode allows listeners to apply parametric EQ, convolution correction, and crossfeed while continuing to bypass the Android framework mixer. Audio is processed at double precision, dithered with TPDF noise, and sent directly to the DAC at its highest supported bit depth.
- *Mode E (Wireless Bluetooth Optimized)*: Routes audio through a low-latency native AudioTrack to BluetoothAudioHAL. The engine queries active codec properties, applies a -1.0\text{ dBFS} pre-attenuation offset to prevent clipping during lossy compression, and dither-downsamples to the codec's negotiated rate to avoid low-quality system resampling.

Laboratory Measurement, Verification, and Testing Protocol

Validating bit-perfect delivery requires verifying the entire playback chain using external hardware analyzers and direct digital captures, rather than relying solely on UI indicators or DAC display panels.

```
+-----------------------------[span_93](start_span)[span_93](end_span)
-------------------------------------------------------------------------
----------------------+
|                                        MEASUREMENT
[span_15](start_span)[span_15](end_span)[span_20](start_span)[span_20]
(end_span)[span_25](start_span)[span_25](end_span)LABORATORY TIERS AND
TOOLSETS                                                |
+------------------+-----------------------------------------------------+--
-----------------------------------------------------+
| Laboratory Tier  | Hardware Instrumentation
Requirement[span_94](start_span)[span_94](end_span)s         |
Software Suites & Tooling                                |
+------------------+-----------------------------------------------------+--
--------[span_95](start_span)[span_95](end_span)----------------------
----------------------+
| Basic Testing    | - Samsung Galaxy S24+ (Exynos 2400)           | -
Android SDK platform-tools (adb logcat, dumpsys)   |
|                  | - USB DAC with sample rate OLED display       | -
Bit-accurate test file suite                       |
|                  |   (e.g., FiiO KA13 / KA17, Moondrop Dawn Pro) | -
Hex editor (010 Editor)                            |
```

```
|                  | - High-performance IEMs / headphones          |
|
|                  |                                                |
|
| Intermediate     | - USB DAC with bit-accurate digital S/PDIF    | -
Audio workstation software (Reaper / Audacity)     |
| Testing          |   optical/coaxial output                      | -
Digital nulling and phase-inversion test tools     |
|                  | - Audio interface with native S/PDIF input    | -
Wireshark with USBPcap capture integration         |
|                  |   (e.g., Focusrite Scarlett 8i6, MOTU M4)     |
|
|                  |                                                |
|
| Advanced         | - Audio Precision APx555 B Series analyzer    | -
APx500 Audio Measurement Software                  |
| Reference Lab    | - RME ADI-2 Pro FS R Black Edition            | -
USB-IF USB 2.0 Electrical & Protocol Test Suite    |
|                  | - Total Phase Beagle USB 480 Protocol         | -
Custom automated bitstream verification scripts    |
|                  |   Analyzer (Hardware bus capture)             |
(Python / NumPy sample-comparison test harness)    |
+------------------+-----------------------------------------------+--
-------------------------------------------------+
```

Core Verification Methodologies

Digital loopback null testing provides direct mathematical verification of bit-perfect delivery. A 24-bit/96 kHz reference file containing white noise, multitone signals, and dynamic acoustic passages is played through the music player engine into a USB DAC configured with digital S/PDIF passthrough. The digital output is captured using a reference studio interface running at the identical clock rate.

The captured recording is aligned with the source file at single-sample precision, inverted in phase, and summed with the original. The null test must sum to absolute digital zero (-\infty\text{ dBFS}) across the entire duration of the audio stream; any residual signal indicates software volume scaling, dither noise injection, sample rate conversion, or active DSP.

Hardware protocol analysis verifies transmission behavior at the bus layer. A hardware USB bus analyzer (e.g., Total Phase Beagle USB 480) is connected inline between the phone and the USB DAC. The analyzer captures USB enumeration packets, UAC2 alternate interface requests, and isochronous data packets during playback initialization. Verification passes when the capture logs confirm that the host issues UAC2 SET_CUR requests matching the source file's native sampling rate, and that raw isochronous payloads contain unaltered PCM words matching the source file.

Android AudioFlinger diagnostic dumps provide operating-system-level confirmation of mixer bypass:

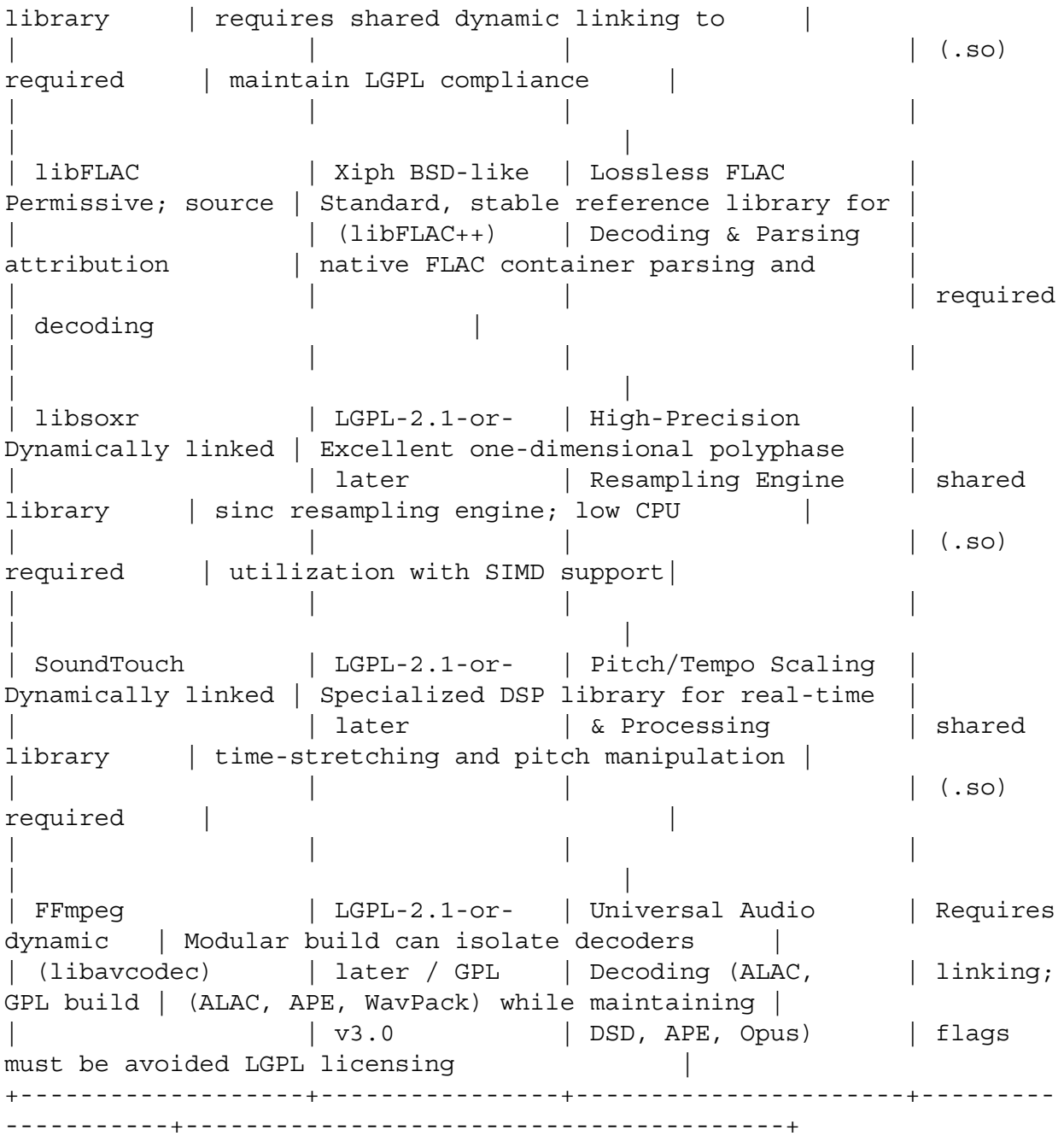
```
adb shell dumpsys media.audio_flinger
```

When using a custom user-space USB driver, the output thread for the external USB device must show 0 active tracks, or reflect that the framework device output is suspended or muted. This confirms that AudioFlinger is not processing the application's audio data. When testing Mode B (AudioMixerAttributes.MIXER_BEHAVIOR_BIT_PERFECT), the diagnostic output must confirm that the active MixerThread has transitioned to a non-resampling direct state.

Open-Source Software Components and Commercial Licensing Strategy

Building an audio engine using established, well-tested open-source libraries can accelerate development while ensuring robust format handling. However, combining these components requires careful licensing analysis to protect intellectual property.

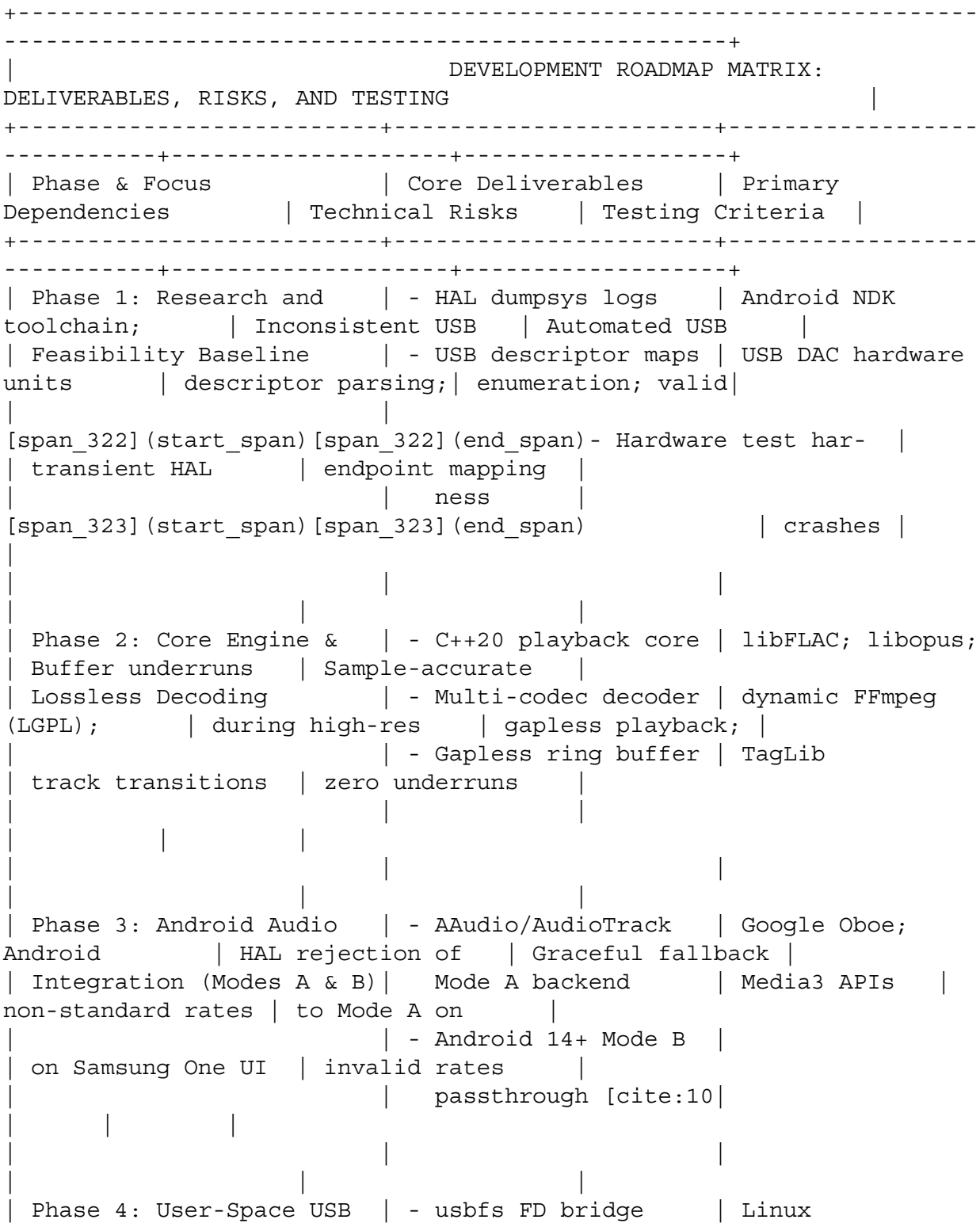
```
+-------------------------------------------------------------------
-------------------------------------------------+
|                                    OPEN-SOURCE AUDIO LIBRARIES AND
LICENSING                                        |
+------------------+---------------+---------------------+---------
----------+----------------------------------------+
| Library / Project | Primary        | Functional Domain    |
Distribution       | Implementation Notes                    |
|                   | License        |                      |
Consideration[span_165](start_span)[span_165](end_span)[span_168](star
t_span)[span_168](end_span)s     | & Suitability
|
+------------------+---------------+---------------------+---------
----------+----------------------------------------+
| Google Oboe       | Apache 2.0     | Native C++ Audio     |
Permissive; safe   | Reference wrapper for AAudio/OpenSL ES;|
|                   |                | Abstraction          | for
proprietary    | ideal foundation for Mode A fallback   |
|                   |                |
[span_70](start_span)[span_70](end_span)           |
comme[span_319](start_span)[span_319](end_span)rcial binaries|
pipeline                  |
|                   |                |                      |
|                                       |
| TinyALSA          | BSD-3-Clause   | Minimal Linux ALSA   |
Permissive; no copy| Lightweight native ALSA library; used  |
|                   |                | User-Space Interface | left
requirements  | for low-level audio device interaction |
|                   |                |                      |
|                                |
|                   |                |                      |
|                                       |
| libusb            | LGPL-2.1-or-   | Cross-Platform USB   |
Dynamically linked | Provides clean asynchronous USB access;|
|                   | later          | Device Access        | shared
```



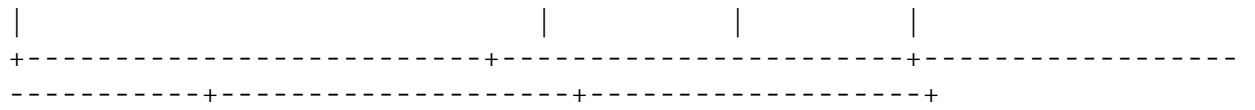
When building a commercial application, libraries licensed under the GNU General Public License (GPL v2/v3) must not be statically linked into proprietary binaries, as the GPL's copyleft terms require open-sourcing the surrounding application codebase.
When using libraries licensed under the GNU Lesser General Public License (LGPL v2.1+), such as libsoxr or FFmpeg, the modules must be compiled as standalone dynamic shared objects (.so) and linked dynamically at runtime. This dynamic linking architecture permits end users to re-link modified versions of the LGPL components, fulfilling license obligations without requiring disclosure of proprietary application source code.

Phased Development Roadmap and Risk Mitigation

This development roadmap organizes implementation into focused engineering phases to systematically manage platform complexity and mitigate technical risks.



usbdevice_fs.h; | Frame drops during | Bit-perfect null |
| Audio Driver (Mode C) | - UAC2 isochronous | Android USB Host API | background state or| test matching; |
| |
[span_325](start_span)[span_325](end_span)[span_326](start_span)[span_326](end_span)[span_327](start_span)[span_327](end_span) transfer engine | | CPU throttling | sample-rate switch|
| | | |
| |
Phase 5: High-Precision	- 64-bit float DSP	libsoxr; KISS FFT /[span_32](start_span)[span_32](end_span)	High CPU usage on	<5% CPU usage
DSP Pipeline (Modes D & E	- PEQ, FIR convolver,	ARM NEON SIMD	long FIR filters;	on Exynos 2400
	crossfeed	vectorization	biquad instability	during active PEQ
[span_328](start_span)[span_328](end_span)	
[span_117](start_span)[span_117](end_span)[span_120](start_span)[span_120](end_span)
Phase 6: DAP-Inspired	- Analog Phase Linear-	Native DSP pipeline;	Aud[span_71](start_span)[span_71](end_span)ible distortion	Verifiable phase
Software Processors	izer; Vinyl acoustic	psychoacoustic filter	from excessive non-	shift; clean har-
[span_331](start_span)[span_331](end_span)
[span_332](start_span)[span_332](end_span)[span_334](start_span)[span_334](end_span)	modeling	designs	linear processing	monic generation
[span_336](start_span)[span_336](end_span)	
Phase 7: Laboratory	- Automated QA suite	Hardware USB analyzer;	Edge-case unplugs
48[span_96](start_span)[span_96](end_span)-hr continuous
[span_33](start_span)[span_33](end_span) Validation & Hardening	- Power/thermal logs	digital S/PDIF capture	causing native JNI	playback with zero
		interface	memory leaks	memory leaks
[span_73](start_span)[span_73](end_span)



Detailed Implementation Phases

Phase 1: Research and Feasibility Baseline

Phase 1 focuses on analyzing the Galaxy S24+ audio environment, capturing USB descriptor maps from representative DACs, and evaluating Audio HAL properties. The engineering team uses Android Debug Bridge utilities (adb shell dumpsys media.audio_policy, dumpsys media.audio_flinger) to document platform routing behaviors under One UI. The primary risk involves inconsistent USB descriptor configurations across DAC models. Success is measured by the reliable parsing of device interfaces and alternate settings across all test units .

Phase 2: Core Audio Engine and Lossless Decoding Infrastructure

Phase 2 establishes the C++20 playback core using the Android NDK. Multi-format container parsing and decoding are integrated using libFLAC, dynamic FFmpeg (libavcodec), libopus, and libwavpack. The team implements double-buffered ring buffers and asynchronous pre-buffering to ensure smooth, sample-accurate gapless playback. Memory allocations within the audio rendering path are strictly avoided to eliminate garbage collection pauses and JNI overhead.

Phase 3: Android Audio Integration (Modes A and B)

Phase 3 implements standard system output routing via Google Oboe and native AudioTrack clients (Mode A). Concurrently, the team integrates Android 14+ AudioMixerAttributes.MIXER_BEHAVIOR_BI[span_321](start_span)[span_321](end_span)T_PERFECT passthrough (Mode B). The engine queries AudioManager.getSupportedMixerAttributes() and applies preferred attributes when matching DAC profiles are identified. The implementation includes automated exception handling and fallback logic to revert smoothly to Mode A if the vendor HAL rejects the requested sample rate.

Phase 4: Dedicated User-Space USB Audio Class Driver (Mode C)

Phase 4 delivers the custom UAC2 driver operating over Linux usbfs. The driver uses UsbDeviceConnection.getFileDescriptor() to claim the interface via USBDEVFS_DISCONNECT_CLAIM and manage streaming via USBDEVFS_SUBMITURB[span_82](start_span)[span_82](end_span)[span_86](start_span)[span_86](end_span)[span_338](start_span)[span_338](end_span)[span_339](start_span)[span_339](end_span)[span_340](start_span)[span_340](end_span)[span_341](start_span)[span_341](end_span) and USBDEVFS_REAPURB. The engine implements UAC2 clock selection, hardware volume control via Feature Unit requests, native DSD bitstreaming (DSD_U32_BE), and DoP v1.1 encapsulation. The primary technical risk involves thread priority starvation when the application transitions to the background; this is addressed by configuring high-priority Linux real-time scheduling threads (SCHED_FIFO).

Phase 5: High-Precision Floating-Point DSP Pipeline (Modes D and E)

Phase 5 implements the 64-bit double-precision floating-point DSP pipeline. The team integrates Direct Form II Transposed biquad filter arrays with automated gain pre-attenuation, uniformly partitioned overlap-save FFT convolution, Linkwitz-Bauer crossfeed, and EBU R128 loudness normalization. The engine incorporates an oversampled true-peak limiter and a TPDF dither generator. Code paths are vectorized using ARM NEON SIMD intrinsics to keep CPU utilization below 5\% on the Exynos 2400 during full multi-band processing .

Phase 6: DAP-Inspired Software Acoustic Processors

Phase 6 develops specialized DSP modules that reproduce desirable analog acoustic characteristics. The team implements an analog DC Phase Linearizer module using allpass filter networks, a Vinyl Acoustic Simulator modeling tonearm resonance and surface drag, and an MSEB-style macro equalization interface. Validation requires confirming measurable phase shifts and controlled low-order harmonic generation without digital clipping .

Phase 7: Laboratory Validation and Hardening

Phase 7 executes the comprehensive laboratory testing plan. The team performs digital loopback null testing against reference studio interfaces, analyzes hardware USB bus traces using protocol sniffers, and conducts thermal and battery profiling on the Galaxy S24+ . The phase concludes with 48-hour continuous stress testing across diverse USB DACs to confirm zero memory leaks, buffer underruns, or unhandled device detachment exceptions.

Architectural Synthesis and Implementation Directives

Deterministic high-resolution audio reproduction on the Samsung Galaxy S24+ (Exynos 2400) requires bypassing the Android audio framework and Samsung One UI post-processing layers using a dedicated, user-space USB Audio Class 2.0 driver. While Android 14's AudioMixerAttributes bit-perfect mode represents an important architectural improvement, vendor HAL inconsistencies make it unsuitable as a sole implementation strategy . Dedicated DAPs provide physical advantages in clock stability, power isolation, and balanced analog amplification that smartphones cannot match without external hardware. However, advanced software capabilities—such as phase linearization, acoustic emulation, and high-precision parametric equalization—can be implemented directly within a native C++ DSP engine.

Final Implementation Directives

A. Must Implement (Essential Core Architecture)

- Native C++20 audio engine built using the Android NDK, integrating libFLAC, FFmpeg (libavcodec), and libopus decoders.
- User-space USB Audio Class 2.0 driver communicating directly via Linux usbfs ioctls (USBDEVFS_SUBMITURB, USBDEVFS_REAPURB), bypassing AudioFlinger, the system Audio HAL, and One UI effects.
- Automatic hardware sample-rate switching using UAC2 Clock Source control requests

across Endpoint 0 (44.1 kHz to 768 kHz).
- Hardware DAC volume control using UAC2 Feature Unit requests, preserving bit depth by keeping software streams unattenuated at 0\text{ dBFS}.
- DSD over PCM (DoP v1.1) encapsulation with alternating 0x05/0xFA sync markers for DSD64 and DSD128 playback.
- Native DSD bitstreaming (DSD_U32_BE) via raw isochronous framing for supported DAC interfaces.
- Sample-accurate gapless playback with double-buffered asynchronous pre-buffering.
- Real-time audio diagnostics interface displaying container format, decoded sample rate, active output mode, and target DAC transmission properties.

B. Implement if Feasible (Advanced Software Additions)

- High-precision parametric equalizer using 64-bit floating-point Direct Form II Transposed biquad filters with automated pre-attenuation gain calculation.
- Uniformly partitioned overlap-save FFT convolution engine for AutoEq headphone profiles and room impulse responses.
- ReplayGain 2.0 / EBU R128 loudness normalization engine paired with an oversampled true-peak limiter.
- Acoustic crossfeed processor using a Linkwitz-Bauer modeled delay-and-shelf network (250\,\mu\text{s} ITD, 700\text{ Hz} ILD).
- Analog DC Phase Linearizer module reproducing low-frequency analog phase characteristics .
- Vinyl Acoustic Simulator modeling tonearm resonance, low-order harmonic warmth, and subtle surface drag acoustics .
- Android 14+ AudioMixerAttributes mode providing a framework-level direct path when low-level USB permissions are withheld.
- Bluetooth headroom optimization engine applying a -1.0\text{ dBFS} pre-attenuation offset and TPDF dithering to minimize inter-sample clipping during lossy A2DP encoding.

C. Do Not Implement or Do Not Claim (Technically Unsupported or Misleading)

- Do not claim bit-perfect transmission over Bluetooth A2DP connections; standard codecs (SBC, AAC, aptX, LDAC) discard data through lossy psychoacoustic compression.
- Do not attempt global operating-system mixer bypass for third-party streaming applications; system-wide bypass requires platform signature keys or custom ROM builds (such as HiBy DTA), which are unavailable to standard user-space applications.
- Do not claim support for Qualcomm aptX Lossless transmission; the target Galaxy S24+ uses an Exynos 2400 SoC that does not integrate Qualcomm Snapdragon Sound hardware.
- Do not attempt to implement isochronous USB audio streaming using the Android Java SDK; UsbRequest does not support isochronous transfers, necessitating native C/C++ driver implementations.
- Do not claim software emulation of analog hardware components; digital processing cannot replicate physical low-phase-noise crystal oscillators, discrete power regulation stages, or balanced analog amplification circuits.

Works cited

1. [Magisk Module] A71 Hidden Mods - XDA Forums, https://xdaforums.com/t/magisk-module-a71-hidden-mods.4236169/ 2. Samsung Galaxy S24 - Notebookcheck Specs und Testsammlung, https://www.notebookcheck.com/Samsung-Galaxy-S24.808742.0.html 3. Samsung Galaxy S24 series review: Is it still worth it in 2025?, https://www.backmarket.co.uk/en-gb/c/samsung/samsung-galaxy-s24-review 4. Android 音频低延时mmap介绍(1) - 稀土掘金, https://juejin.cn/post/7324151621297831971 5. Feature Request: Bit-Perfect USB Audio Passthrough (Android 14+), https://github.com/OxygenCobalt/Auxio/issues/1210 6. AudioMixerAttributes | API reference - Android Developers, https://developer.android.com/reference/android/media/AudioMixerAttributes 7. AudioMixerAttributes.Builder.SetMixerBehavior(AudioMixerBehavior, https://learn.microsoft.com/hu-hu/dotnet/api/android.media.audiomixerattributes.builder.setmixerbehavior?view=net-android-37.0 8. Various playback issues on Pixel 7a with ... - GitHub, https://github.com/google/oboe/issues/2343 9. NW-WM1ZM2 Specifications | Walkman - Sony, https://www.sony.com/en-cd/electronics/walkman/nw-wm1zm2/specifications 10. libusbhost/usbhost.c - platform/system/core - Android GoogleSource, https://android.googlesource.com/platform/system/core/+/android17-release/libusbhost/usbhost.c 11. USB host overview | Connectivity - Android Developers, https://developer.android.com/develop/connectivity/usb/host 12. decent-player/docs/driver/02-investigation-journey.md at main, https://github.com/Ma145/decent-player/blob/main/docs/driver/02-investigation-journey.md 13. Diff - kernel/msm - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/msm/+/1da177e4c3f41524e886b7f1b8a0c1fc7321cac2%5E%21/ 14. Is Samsung Seamless Codec a Bluetooth Audio Upgrade?, https://www.free-codecs.com/news/is-samsung-seamless-codec-a-bluetooth-audio-upgrade.htm 15. Press Releases | Sony Asia Pacific, https://www.sony-asia.com/pressrelease?prName=sony-remasters-signature-series 16. USB: low latency (< 1ms) with interrupt transfer and raw HID, https://stackoverflow.com/questions/27976992/usb-low-latency-1ms-with-interrupt-transfer-and-raw-hid 17. Sony updates its Signature Series hi-res Walkman with new features, https://www.digitaltrends.com/home-theater/sony-signature-series-walkman-nw-wmzm2-nw-wm1am2-hi-res-portables/ 18. [UNOFFICIAL][ROM]BETA LineageOS 20 for SM-T715 - XDA Forums, https://xdaforums.com/t/unofficial-rom-beta-lineageos-20-for-sm-t715-october-11-2026.4606053/page-2 19. Analysis search | TechInsights, https://techinsights.com/search?SearchResult=17 20. Search | Device Specs - The Largest Phone Specs Database, https://phonedb.net/index.php?m=device&s=query&d=detailed_specs&os=41 21. Galaxy Buds3 Silver | Audio Sound | Samsung Lebanon, https://www.samsung.com/lb/audio-sound/galaxy-buds/galaxy-buds3-silver-sm-r530nzaamea/ 22. Business | Galaxy Buds3 | SM-R530NZWAXME | Samsung Malaysia, https://www.samsung.com/my/business/watches/galaxy-watch/galaxy-buds3-white-sm-r530nzwaxme/ 23. The Linux-USB Host Side API - The Linux Kernel documentation, https://docs.kernel.org/driver-api/usb/usb.html 24. Samsung Galaxy S24 FE - Notebookcheck.fr, https://www.notebookcheck.biz/Samsung-Galaxy-S24-FE.904045.0.html 25. Samsung

Electronics 2024 Sustainability Report,
https://www.samsung.com/global/sustainability/media/pdf/Samsung_Electronics_Sustainability_Report_2024_ENG.pdf 26. Samsung Unveils All New Galaxy Buds4 Series with Ultimate Sound,
https://news.samsung.com/sg/samsung-unveils-all-new-galaxy-buds4-series-with-ultimate-sound 27. The Ultimate Sound tuning and DSP guide for M2 Walkman(Offline,
https://cdn.head-fi.org/a/12484439.pdf 28. รีวิว (Review) Samsung Galaxy Note 8 สมาร์ทโฟนเรือธงรุ่นใหญ่ใหม่, https://www.thaimobilecenter.com/review/samsung-galaxy-note8.asp 29. Maximum Audio Quality on S24 FE with Hi-Res Music? - Reddit,
https://www.reddit.com/r/samsunggalaxy/comments/1l1jnud/maximum_audio_quality_on_s24_fe_with_hires_music/ 30. รีวิว Samsung Galaxy S8 และ Galaxy S8+ ยอดสมาร์ทโฟนเรือธงตัวท็อป,
https://www.thaimobilecenter.com/review/samsung-galaxy-s8.asp 31. Business | Galaxy Buds3 Pro | SM-R630NZWAEUE - Samsung,
https://www.samsung.com/lv/business/audio-sound/galaxy-buds/galaxy-buds3-pro-sm-r630nzwaeue/ 32. Uhd audio greedy out with the xm5 headphones : r/S24Ultra - Reddit,
https://www.reddit.com/r/S24Ultra/comments/1o6b2o7/uhd_audio_greedy_out_with_the_xm5_headphones/ 33. Apple music preferred over Tidal and Qobuz for offline use - Facebook,
https://www.facebook.com/groups/headfi.org/posts/10163775728974736/ 34. Reviews by Ichos | Page 3 - Head-Fi.org,
https://www.head-fi.org/showcase/authors/ichos.498275/reviews?order=rating&direction=desc&page=3 35. Best SAM Broadcaster Pro Alternatives & Competitors - SourceForge,
https://sourceforge.net/software/product/SAM-Broadcaster-Pro/alternatives 36. Inside an Embedded Linux Audio Product: USB DSD Quirks, Kernel,
https://www.grinaldssolutions.com/blog/embedded-linux-audio-product-development.html 37. DSD native 與DoP 差在哪？Linux MPD 設定與bit-perfect 驗證,
https://lingyinaudio.com/differences-between-dsd-native-and-dop/ 38. WM1AM2 Walkman® Digital Media Player - Sony Centre,
https://centresdirect.co.uk/p-41689-wm1am2-walkman-digital-media-player.aspx?xp=GFCD 39. UsbConstants | API reference - Android Developers,
https://developer.android.com/reference/android/hardware/usb/UsbConstants 40. The basics of USB device development using the Android framework,
https://www.embedded.com/the-basics-of-usb-device-development-using-the-android-framework/ 41. USBIsochronousOutTransferPac,
https://developer.mozilla.org/en-US/docs/Web/API/USBIsochronousOutTransferPacket 42. 經典音響交流社, https://www.facebook.com/groups/843368775764250/posts/26132585106415935/ 43. SONY NW-WM1Z M2 / WM1A M2 - Head-Fi.org,
https://www.head-fi.org/threads/sony-nw-wm1z-m2-wm1a-m2.962054/ 44. 不少人花錢買的DSD，可能根本沒正確送到你的DAC。而且設定錯誤,
https://www.facebook.com/groups/843368775764250/posts/27605578405783257/ 45. Доработка MPD (C++, Linux) - Конференция iXBT.com,
https://forum.ixbt.com/topic.cgi?id=94:103 46. Buy the FiiO M33 R2R Hi-Res DAP - Headphone Zone, https://www.headphonezone.in/products/fiio-m33-r2r 47. FiiO M27: Flagship Portable Digital Audio Player - Manuals.plus,
https://manuals.plus/m/364c5be7373a6a793c8375d568c0eb7370f476dc240b5b7e47381a82dc9fc403 48. why do Dolby's panoramic sound need to wait? M11 Plus LTD ... - FiiO,
https://www.fiio.com/newsinfo/521024.html?templateId=1133604 49. High-Res Player - FiiO,
https://fiio-shop.de/EN/Portable-Audio/High-Res-Player/ 50. FiiO M15S Hi-Res Portable Digital Audio Player - B&H,

https://www.bhphotovideo.com/c/product/1759242-REG/fiio_m15s_5_bluetooth_portable.html 51. FiiO M27 - Fidelity Magazine, https://www.fidelity-magazine.com/fiio-m27/ 52. FiiO M21 Review - MOONSTAR Reviews », https://moonstarreviews.net/fiio-m21-review.htm 53. Sony NW-ZX707 Walkman Hi-Res Portable Audio Player, https://www.headphonezone.in/products/sony-nw-zx707 54. Sony rejuvenates Signature Series | Porta-Fi™, https://porta-fi.com/sony-rejuvenates-signature-series/